

UNIT-2 CONVOLUTIONAL NEURAL NETWORKS

1. Introduction to Convolutional Neural Networks

1.1 What is a Convolutional Neural Network (CNN)?

A Convolutional Neural Network (CNN) is a specialized type of deep neural network designed primarily to process image and visual data. CNNs are particularly effective for images because they can automatically learn important visual features such as edges, textures, shapes, and objects directly from the input image.

In a traditional neural network, an image may need to be converted into a large vector of pixel values before processing. This can result in a very large number of parameters and high computational requirements. CNNs overcome this problem by using convolutional layers, which examine small regions of an image and learn local patterns.

CNNs also use pooling layers to reduce the spatial size of feature maps while retaining important information. Through a sequence of convolution, activation, pooling, and classification operations, a CNN gradually transforms the raw image into meaningful high-level representations.

The major advantage of CNNs is that they can perform automatic feature extraction, eliminating the need to manually design features for many image-processing tasks.

“A Convolutional Neural Network (CNN) is a deep neural network that uses convolution operations and related layers to automatically learn hierarchical features from image or visual data and use those features for tasks such as classification and recognition.”

Important Characteristics of CNN

1. Automatic Feature Learning
2. Local Feature Extraction
3. Hierarchical Feature Learning
4. Parameter Sharing
5. Preservation of Spatial Information
6. Reduced Computational Complexity

Applications

CNNs are widely used in image classification, object detection, face recognition, medical image analysis, image segmentation, handwriting recognition, autonomous vehicles, and security systems.

1.2 Why Do We Need Convolutional Neural Networks?

Images contain a large number of pixels, and directly processing all these pixels using a traditional Artificial Neural Network can become computationally expensive.

Limitations of Traditional ANN for Image Processing

1. **Large Image Inputs:** Images may contain thousands or millions of pixels. When these pixels are directly connected to neurons in a fully connected network, the input size becomes very large.
2. **Too Many Parameters:** A fully connected network may require a separate weight for every connection between input pixels and neurons. This leads to a very large number of parameters.
3. **High Computational Cost:** A large number of parameters increases the memory and computational requirements during training and prediction.

4. **Cannot Efficiently Capture Local Features:** Important visual patterns such as edges, corners, and textures usually occur within local regions of an image. A traditional fully connected network does not naturally exploit this local structure.
5. **Risk of Overfitting:** A network containing a very large number of parameters can memorize training data instead of learning general patterns, increasing the possibility of overfitting.

How CNNs Address These Problems

- **Convolution:** Extracts local features using small filters.
- **Parameter Sharing:** The same filter weights are reused across different image locations.
- **Local Connectivity:** Each neuron examines only a small region of the input.
- **Pooling:** Reduces the spatial dimensions of feature maps.
- **Hierarchical Learning:** Learns increasingly complex features through multiple layers.

Therefore, CNNs can process image data more efficiently while automatically learning useful visual representations.

Comparison of Traditional ANN and CNN

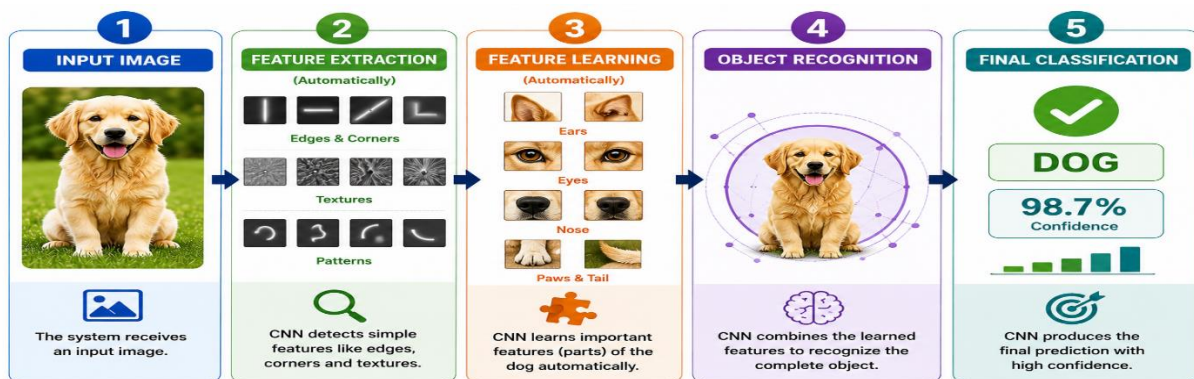
Traditional ANN	CNN
Treats image pixels mainly as individual input values	Exploits spatial structure of images
Large number of parameters	Fewer parameters through parameter sharing
Does not naturally focus on local regions	Uses local connectivity
Manual feature extraction may be required	Automatically learns features
Less suitable for large images	Well suited for image and visual data

1.3 Basic Working Principle of CNN

A CNN processes an image through a sequence of stages. At each stage, the network extracts and transforms useful information from the input.

The general working process is:

Input Image → Feature Extraction → Feature Learning → Object Recognition → Final Classification



Step 1: Input Image

The CNN receives an image as input. The image is represented numerically using pixel values.

For example:

- A grayscale image contains a single channel.
- An RGB image contains three channels: Red, Green, and Blue.

Step 2: Feature Extraction

The convolution layers apply filters or kernels to different regions of the image.

These filters learn basic visual features such as:

- Edges
- Lines
- Corners
- Textures

Step 3: Feature Learning

As the data passes through deeper layers, the CNN combines simple features to learn more complex representations.

For example:

Edges → Textures → Shapes → Object Parts → Complete Objects

Thus, the network gradually develops a hierarchical understanding of the image.

Step 4: Object Recognition

The learned high-level features are used to identify the object or pattern present in the image.

For example, after learning edges, shapes, and object parts, the network may recognize that the image contains a cat, car, dog, or person.

Step 5: Final Classification

The extracted features are passed to the later layers of the network, where the final class or prediction is produced.

For example:

Input Image → CNN → "Cat"

The CNN therefore learns to transform raw pixel information into a meaningful prediction without requiring manual feature extraction.

2. Components of CNN Architecture

A Convolutional Neural Network consists of several interconnected components, where each component performs a specific function in processing an image. The input image is progressively transformed into feature representations and finally into a prediction or classification result.

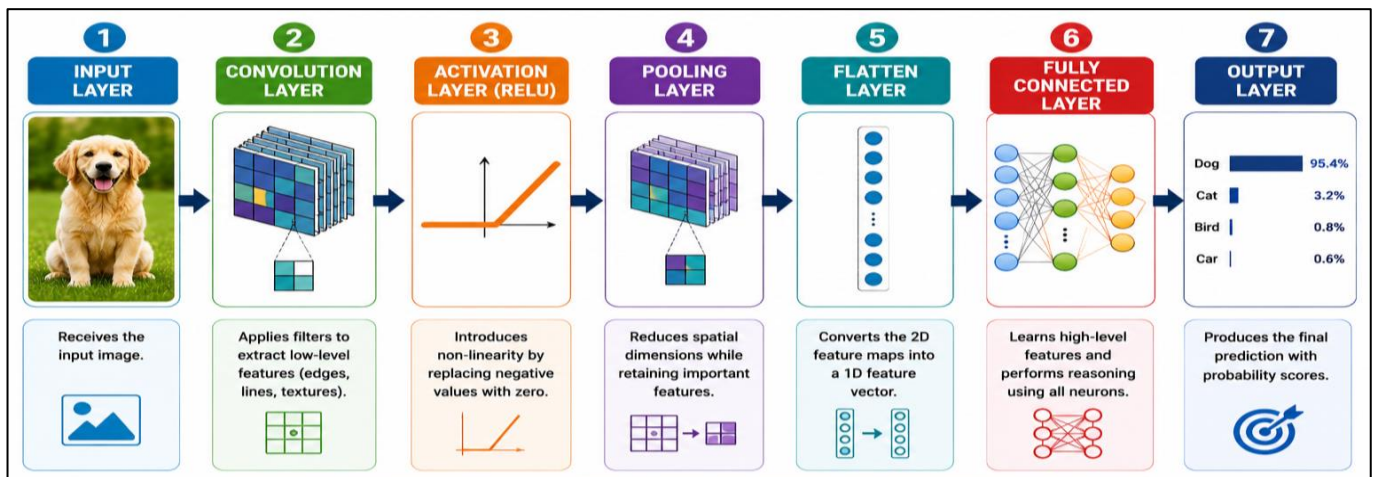
The major components of a typical CNN are:

1. Input Layer
2. Convolution Layer
3. Filter (Kernel)
4. Convolution Operation
5. Stride
6. Padding
7. Feature Map
8. ReLU Activation Layer
9. Pooling Layer
10. Flatten Layer
11. Fully Connected Layer
12. Output Layer

These components work together in sequence to extract useful features from an image and produce the final output.

Overall CNN Flow

Input Image → Convolution → ReLU → Pooling → Flatten → Fully Connected Layer → Output



2.1 Input Layer:

The Input Layer is the first layer of a Convolutional Neural Network. It receives the input image and passes the image data to the subsequent CNN layers for processing.

Unlike the convolution and pooling layers, the input layer does not perform feature extraction. Its main purpose is to provide the image in a suitable numerical form to the network.

Functions of Input Layer:

- **Receives the Input Image:** Accepts the image that needs to be processed by the CNN.
- **Represents Image Data:** Represents the image using numerical pixel values that can be processed by the network.

- **Supports Different Image Types:** Can receive both grayscale and color images.
- **Preserves Image Structure:** Maintains the height, width, and channel information of the input image.
- **Passes Data to the Next Layer:** Sends the image representation to the first convolution layer for feature extraction.

Image Representation: An image is represented as a multidimensional array based on its height, width, and number of channels.

1. Grayscale Image: A grayscale image contains a single channel.

- Each pixel represents an intensity value.
- A commonly used intensity range is 0 to 255.
- 0 represents black.
- 255 represents white.
- Values between 0 and 255 represent different shades of gray.

A grayscale image can be represented as: $H \times W \times 1$

where: $H \rightarrow$ Height, $W \rightarrow$ Width, $1 \rightarrow$ Number of channels

2. RGB Image: A color image commonly uses the RGB (Red, Green, Blue) representation.

It contains three channels:

- Red (R)
- Green (G)
- Blue (B)

Therefore, an RGB image can be represented as: $H \times W \times 3$

NOTE: The Input Layer does not perform feature extraction or complex mathematical operations. It provides the image data in a suitable numerical representation to the subsequent CNN layers.

2.2 Convolution Layer:

The Convolution Layer is the core building block of a Convolutional Neural Network. It extracts important visual features from an input image by applying small filters, also called kernels, over different regions of the image.

Instead of connecting every neuron to the entire image, the convolution layer examines small local regions and learns patterns present in those regions. The extracted information is represented as feature maps.

Functions of the Convolution Layer:

- **Feature Extraction:** Extracts useful visual features from the input image.
- **Local Pattern Detection:** Detects patterns such as Edges, Lines, Corners, Textures
- **Feature Map Generation:** Produces feature maps after applying filters to the input.
- **Spatial Information Preservation:** Maintains the spatial relationship between pixels and detected features.
- **Automatic Feature Learning:** Learns suitable filters during the training process instead of requiring manually designed filters.
- **Hierarchical Feature Learning:** In deeper convolution layers, the network can combine simpler features to identify more complex patterns.

How the Convolution Layer Works:

The convolution layer processes an image using a small filter or kernel.

The basic process is:

1. The input image is provided to the convolution layer.
2. A small filter (kernel) is placed over a local region of the image.
3. The filter values are combined with the corresponding pixel values.
4. The resulting values are summed to produce an output value.
5. The filter moves to another region of the image.
6. The process is repeated over the required regions.
7. The resulting values form a feature map.

Example:

Consider an image containing a vertical edge.

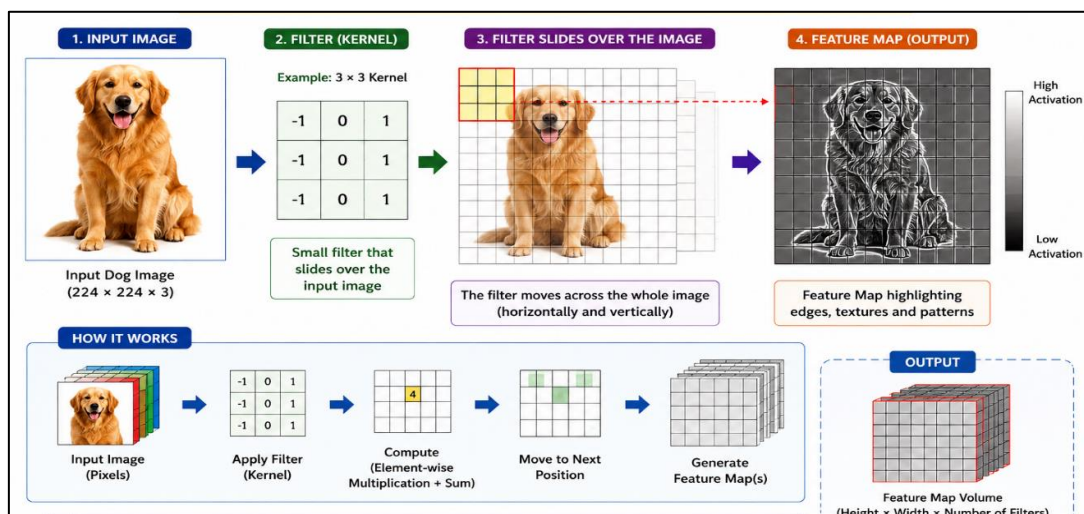
- A filter designed to respond to vertical patterns is applied to different regions of the image.
- Regions containing the vertical edge produce stronger responses.
- These responses form a feature map highlighting the detected edge.

Different filters can learn different types of features. Therefore, applying multiple filters to the same image produces multiple feature maps.

Importance of the Convolution Layer:

The convolution layer is important because it:

- Reduces the need for manual feature extraction.
- Learns useful visual patterns automatically.
- Processes local regions of an image efficiently.
- Provides feature representations for subsequent CNN layers.
- Forms the foundation for extracting increasingly complex features in deeper layers.



2.3 Filter (Kernel):

A filter, also called a kernel, is a small matrix of numerical values used by the convolution layer to detect specific visual features in an image. The filter moves across different regions of the input image and performs the convolution operation to identify patterns.

Characteristics of a Filter:

- It is a small matrix of numerical values.
- Common filter sizes include: 3×3 , 5×5 , 7×7
- It moves across the input image during convolution.
- It examines a small local region at a time.
- It can detect visual patterns such as: Edges, Lines, Corners, Textures
- Different filters can learn different types of features.
- Each filter produces a corresponding feature map.

Role of a Filter:

The main purpose of a filter is to detect a particular pattern or feature in an image.

For example:

- One filter may respond strongly to horizontal edges.
- Another filter may detect vertical edges.
- Another filter may detect a particular texture or pattern.

During CNN training, the values in the filters are learned automatically from the training data. Therefore, the network learns which filters are useful for extracting features from the input images.

Filter and Feature Map:

The relationship can be represented as:

Input Image + Filter (Kernel) $\rightarrow$ Convolution Operation $\rightarrow$ Feature Map

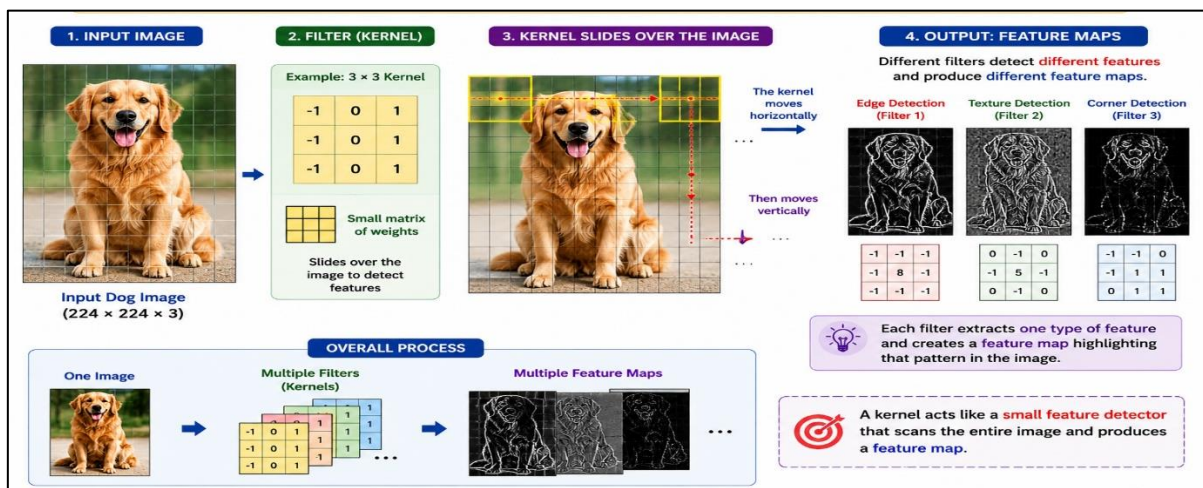
If multiple filters are used in a convolution layer:

Input Image $\rightarrow$ Filter 1 $\rightarrow$ Feature Map 1

Input Image $\rightarrow$ Filter 2 $\rightarrow$ Feature Map 2

Input Image $\rightarrow$ Filter 3 $\rightarrow$ Feature Map 3

Thus, different filters allow the CNN to extract different types of visual information from the same input image.



2.4 Convolution Operation:

The Convolution Operation is the process of applying a filter to different regions of an input image to extract useful visual features. It is the fundamental operation performed by the convolution layer.

During convolution, the filter moves across the image and performs calculations between the filter values and the corresponding pixel values. The results are used to create a feature map.

Steps in Convolution Operation:

1. **Place the Filter:** Place the filter over a small region of the input image.
2. **Perform Element-wise Multiplication:** Multiply each value of the filter with the corresponding pixel value of the image.
3. **Calculate the Sum:** Add all the resulting multiplied values.
4. **Generate an Output Value:** The calculated sum becomes one value in the output feature map.
5. **Move the Filter:** Move the filter to the next position according to the specified stride.
6. **Repeat the Process:** Continue the same operation until the filter has covered the required regions of the image.
7. **Generate the Feature Map:** The collection of output values forms the feature map.

Mathematical Representation:

For a two-dimensional input, the convolution operation can be represented as:

$$Output(i, j) = \sum_m \sum_n Input(i + mj + n) \times Filter(m, n)$$

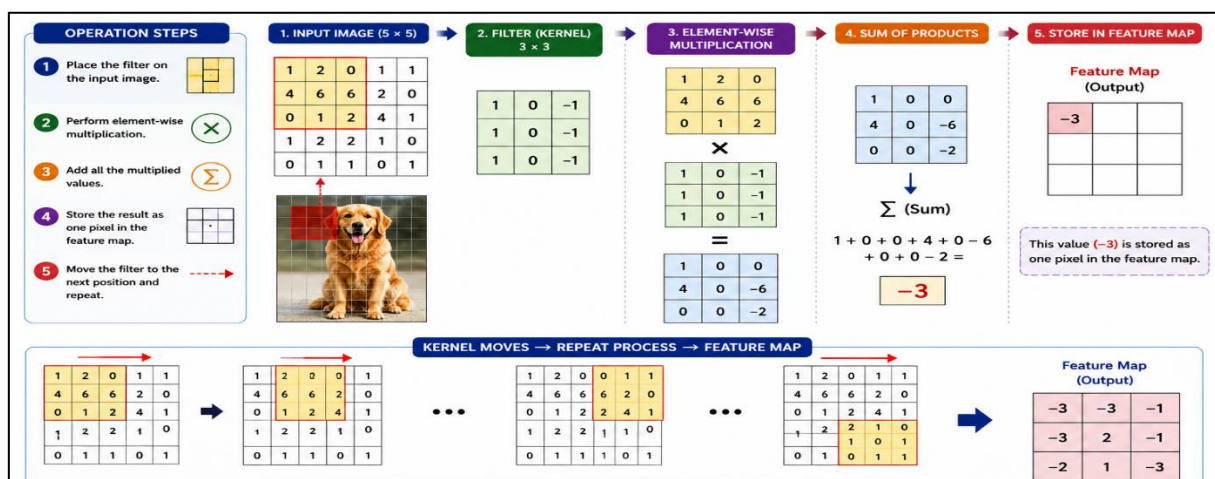
where:

- Input → Input image or feature map
- Filter → Kernel applied to the input
- Output → Resulting feature map
- i, j → Position of the filter on the input
- m, n → Row and column positions within the filter/kernel
- $Input(i + mj + n)$ → Input value corresponding to the current filter position
- $Filter(m, n)$ → Filter value at row m and column n

NOTE: At each position (i, j) , multiply the input values with the corresponding filter values, add all the products, and store the result as one value in the output feature map.

Importance of Convolution Operation:

- Extracts useful features from images.
- Detects local patterns such as edges, lines, and textures.
- Produces feature maps for further processing.
- Allows the same filter to detect a feature at different positions in an image.
- Forms the basic feature-extraction mechanism of CNNs.



2.5 Stride:

Stride is the number of pixels by which a filter (kernel) moves across the input image during the convolution operation. It determines how frequently the filter samples different regions of the input.

Types of Stride:

Stride = 1

- The filter moves one pixel at a time.
- More overlapping regions of the image are processed.
- Produces a relatively larger feature map.
- Preserves more spatial information.

Stride = 2

- The filter moves two pixels at a time.
- Fewer regions of the image are processed.
- Produces a smaller feature map.
- Reduces the spatial dimensions of the output.

Effect of Stride:

- Smaller stride → Larger feature map
- Larger stride → Smaller feature map
- Increasing the stride reduces the number of convolution operations.
- Therefore, a larger stride can reduce computational requirements but may discard some spatial information.

2.6 Padding:

Padding is the process of adding extra pixels, usually zeros, around the border of an input image before performing the convolution operation. It is used to control the spatial size of the output feature map and to preserve information near the image boundaries.

Why is Padding Used?

- **Preserves Boundary Information:** Pixels near the edges of an image can be included effectively in the convolution operation.
- **Prevents Excessive Reduction in Size:** Without padding, the spatial dimensions of the feature map become smaller after convolution.
- **Allows Filters to Process Edge Pixels:** Adding a border allows the filter to cover regions near the image boundaries.
- **Helps Preserve Spatial Information:** Important features occurring near the edges are less likely to be lost.

Without Padding:

- The filter is applied only where it completely fits inside the input.
- The output feature map becomes smaller.
- Border pixels receive less attention than pixels in the center.
- Repeated convolution operations can significantly reduce the spatial dimensions.

With Padding:

- Additional pixels are added around the image border.

- The filter can also operate near the image boundaries.
- The output spatial dimensions can be maintained depending on the padding and stride.
- More information near the image boundaries can be retained.

2.7 Feature Map

A Feature Map is the output produced after applying a filter (kernel) to an input image or feature map during the convolution operation. It represents the locations where the feature detected by the filter is present.

Role of a Feature Map:

- Shows the response of a particular filter to the input image.
- Highlights the regions where the filter detects its corresponding feature.
- Represents useful visual patterns extracted from the input.
- Provides the extracted information to the subsequent CNN layers.

Important Characteristics:

- Each filter produces one feature map.
- Different filters can detect different types of features.
- A single convolution layer can therefore produce multiple feature maps.
- Feature maps preserve information about the location of detected features.
- As the network becomes deeper, feature maps can represent increasingly complex patterns.

Example:

Suppose three different filters are applied to the same image:

- Filter 1 → Detects edges → Feature Map 1
- Filter 2 → Detects textures → Feature Map 2
- Filter 3 → Detects another visual pattern → Feature Map 3

The filter and feature map are closely related but represent different things:

Feature Maps in Deeper CNN Layers:

CNNs learn features progressively through different layers:

- **Early layers:** Edges, lines, and simple patterns
- **Middle layers:** Textures, shapes, and combinations of patterns
- **Deeper layers:** Object parts and complex structures

Therefore, feature maps provide the intermediate representations through which a CNN gradually learns meaningful information from an image.

2.8 ReLU Activation Layer:

The ReLU (Rectified Linear Unit) Activation Layer is applied after the convolution operation. It introduces non-linearity into the CNN and transforms the values in the feature map before the data is passed to the next layer.

ReLU Function:

ReLU is defined as:

$$f(x) = \max(0, x)$$

$$\text{If } x > 0, \text{ then: } f(x) = x$$

$$\text{If } x \leq 0, \text{ then: } f(x) = 0$$

Therefore, ReLU retains positive values and converts negative values to zero.

2.9 Pooling Layer:

The Pooling Layer reduces the spatial size of the feature maps while retaining the important information extracted by the convolution layer. It is generally applied after the activation layer in a CNN.

Functions of Pooling Layer:

- **Reduces Spatial Dimensions:** Reduces the height and width of feature maps.
- **Reduces Computational Requirements:** Smaller feature maps require fewer computations in subsequent layers.
- **Retains Important Information:** Keeps the most significant responses from local regions.
- **Provides Some Robustness:** Helps the network handle small changes in the position of features in an image.
- **Reduces the Amount of Data:** Decreases the number of values that need to be processed by later layers.

Types of Pooling:

The two commonly used pooling methods are:

1. Max Pooling:

Max Pooling selects the maximum value from a small region of the feature map.

For example, consider a 2×2 region:

$$\begin{bmatrix} 1 & 5 \\ 2 & 3 \end{bmatrix}$$

The maximum value is: 5

Therefore, the output for that region is 5.

Purpose of Max Pooling

- Retains the strongest feature response.
- Reduces the spatial dimensions of the feature map.
- Helps preserve prominent features such as strong edges.

2. Average Pooling

Average Pooling calculates the average value of all the values in a selected region of the feature map.

For example:

$$\begin{bmatrix} 1 & 5 \\ 2 & 4 \end{bmatrix}$$

The average is: $\frac{1+5+2+4}{4} = 3$

Therefore, the output for that region is 3.

Purpose of Average Pooling

- Reduces the spatial dimensions of feature maps.
- Provides an average representation of the local region.
- Produces smoother feature representations.

2.10 Flatten Layer:

The Flatten Layer converts the multidimensional feature maps produced by the convolution and pooling layers into a single one-dimensional vector. This vector is then provided as input to the Fully Connected Layer for final classification.

Need for Flattening:

Convolution and pooling layers produce feature maps in multidimensional form, such as:

$$\text{Height} \times \text{Width} \times \text{Channels}$$

However, a fully connected layer expects its input in the form of a one-dimensional vector. The Flatten Layer performs this conversion.

Working of Flatten Layer:

The Flatten Layer performs the following operation:

1. Receives the multidimensional feature maps from the previous layer.
2. Arranges all the values in a single sequence.
3. Converts the multidimensional representation into a one-dimensional vector.
4. Passes the resulting vector to the Fully Connected Layer.

Example

Suppose the feature maps have dimensions:

$$4 \times 4 \times 3$$

The total number of values is: $4 \times 4 \times 3 = 48$

After flattening: $4 \times 4 \times 3 \rightarrow 48 \times 1$

Thus, the $4 \times 4 \times 3$ feature maps become a one-dimensional vector containing 48 values.

Importance of Flatten Layer:

- Converts multidimensional feature maps into a one-dimensional form.
- Provides the required input format for the fully connected layer.
- Transfers the features learned by convolution and pooling layers to the classification stage.
- Does not perform feature extraction or learning; it only changes the shape of the data.

2.11 Fully Connected Layer:

The Fully Connected (FC) Layer receives the one-dimensional feature vector produced by the Flatten Layer and uses the learned features to perform the final classification or prediction. In this layer, every neuron is connected to all the inputs from the previous layer.

Functions of Fully Connected Layer:

- **Receives Flattened Features:** Takes the one-dimensional vector generated by the Flatten Layer.
- **Combines Learned Features:** Combines the features extracted by the earlier convolution and pooling layers.
- **Learns Relationships:** Learns relationships among the extracted features to support classification.
- **Learns High-Level Patterns:** Uses the extracted features to identify high-level patterns relevant to the target classes.
- **Produces Class Scores:** Generates scores corresponding to the possible output classes.
- **Passes Information to the Output Layer:** Sends the resulting class scores to the final output layer.

Working of Fully Connected Layer:

The flattened vector contains the features learned by the earlier CNN layers. Each fully connected neuron processes these inputs using weights and bias and produces an output.

The resulting outputs represent the class scores, which are then passed to the output layer for the final prediction.

Importance of Fully Connected Layer:

- Connects the feature-extraction part of CNN with the classification part.
- Combines features learned at different stages of the network.
- Helps make the final classification decision.
- Converts learned feature representations into class-related scores.

2.12 Output Layer

The Output Layer is the final layer of a Convolutional Neural Network. It receives the information from the preceding fully connected layer and produces the final prediction or classification result.

Functions of the Output Layer

- **Produces the Final Output:** Generates the final result of the CNN.
- **Performs Classification:** Determines the class to which the input image belongs.
- **Uses Class Scores:** Receives the class scores from the Fully Connected Layer.
- **Provides the Prediction:** Produces the final predicted class or output.

Example:

Consider a CNN designed to classify images into three classes: Cat, Dog, Car

The Fully Connected Layer produces scores for the three classes. The Output Layer uses these scores to produce the final prediction.

For example:

Cat $\rightarrow$ 0.10 Dog $\rightarrow$ 0.85 Car $\rightarrow$ 0.05

The CNN predicts: Dog because it has the highest output score.

Output Depends on the Task:

The form of the output depends on the type of problem being solved.

- **Binary Classification:** Produces an output representing one of two classes.
- **Multi-class Classification:** Produces outputs corresponding to multiple classes.
- **Other CNN Tasks:** The output structure can vary depending on the application.

3. ReLU Layer

The Rectified Linear Unit (ReLU) is an activation function widely used in Convolutional Neural Networks. It is generally applied after the convolution operation to introduce non-linearity and enable the network to learn complex patterns. In CNNs, ReLU transforms the feature-map values before they are passed to subsequent layers.

3.1 ReLU (Rectified Linear Unit):

ReLU is an activation function that produces zero for negative input values and retains positive input values unchanged.

Formula $f(x) = \max(0, x)$ OR

$$f(x) = \begin{cases} 0, & x \leq 0 \\ x, & x > 0 \end{cases}$$

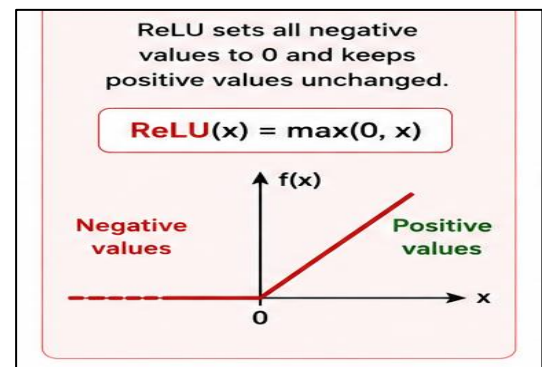
Therefore:

- If $x < 0$, output = 0
- If $x = 0$, output = 0
- If $x > 0$, output = x

Example:

For the input: $x = [-1, 4, -2, 0, 3, 6]$

After applying ReLU: $f(x) = [0, 4, 0, 0, 3, 6]$



Functions of ReLU in CNN:

- **Introduces Non-linearity:** Helps CNNs learn complex patterns and relationships.
- **Removes Negative Responses:** Converts negative feature-map values to zero.
- **Retains Positive Responses:** Preserves positive feature responses.
- **Produces Sparse Activations:** Many values become zero after activation.
- **Computationally Efficient:** The function is simple to calculate.
- **Supports Deep Networks:** Helps successive CNN layers learn increasingly complex representations.

3.2 Dying ReLU Problem:

The Dying ReLU Problem is a limitation of ReLU in which a neuron may become inactive and stop learning. This happens mainly because ReLU has a zero gradient for negative input values.

Steps of the Dying ReLU Problem:

Step 1: Calculate the Weighted Sum

The neuron calculates: $z = w^T x + b$

where $w^T x$ represents the weighted sum of the inputs and b is the bias.

Step 2: Weighted Sum Becomes Negative

Suppose: $z < 0$

The neuron has entered the negative region of the ReLU function.

Step 3: Apply ReLU

Since $z < 0$: $y = \text{ReLU}(z) = 0$

The neuron produces zero output.

Step 4: ReLU Derivative Becomes Zero

For negative input: $\text{ReLU}'(z) = 0$

Therefore, no gradient is passed through the ReLU activation.

Step 5: Gradient Becomes Zero

During backpropagation: $\frac{\partial L}{\partial w_i} = 0$

The neuron therefore receives no effective gradient for updating its weights.

Step 6: Weight Update Becomes Zero

The weight update using gradient descent is:

$$w_i^{new} = w_i^{old} - \eta \frac{\partial L}{\partial w_i}$$

where:

- $w_i^{old} \rightarrow$ current/old value of the weight before the update
- $w_i^{new} \rightarrow$ new/updated value of the weight after the update
- $\eta \rightarrow$ learning rate
- $\frac{\partial L}{\partial w_i} \rightarrow$ gradient of the loss with respect to the weight

For a dead ReLU neuron: $\frac{\partial L}{\partial w_i} = 0$

Therefore: $w_i^{new} = w_i^{old} - \eta(0)$

Thus, the weights remain unchanged through this gradient path.

Step 7: Neuron Stops Learning

If the neuron continues to receive negative inputs, it repeatedly produces zero and receives zero gradients. The neuron may therefore become permanently inactive.

Such a neuron is called a dead neuron, and this condition is known as the Dying ReLU Problem.

Causes of Dying ReLU:

The problem may occur due to:

- **Large learning rate:** Large weight updates may push a neuron into the negative region.
- **Negative weighted inputs:** A neuron may consistently receive negative values.
- **Unfavorable weight updates:** Training may cause the neuron to remain in the negative region.

Effects of Dying ReLU:

- The neuron becomes inactive.
- It continuously produces zero output.
- It stops contributing useful information.
- Its weights may stop receiving effective updates.
- If many neurons become inactive, the learning capacity of the network can be reduced.

4. ELU and SELU Activation Functions

ReLU is widely used because of its simplicity and computational efficiency. However, the Dying ReLU Problem can occur because ReLU produces zero output and has zero gradient for negative inputs. ELU and SELU are alternative activation functions that address some of these limitations.

4.1 ELU – Exponential Linear Unit:

Exponential Linear Unit (ELU) is an activation function that behaves like ReLU for positive inputs but produces negative values for negative inputs.

Formula
$$f(x) = \begin{cases} x, & x > 0 \\ \alpha(e^x - 1), & x \leq 0 \end{cases}$$

where:

- $x \rightarrow$ Input value
- $\alpha \rightarrow$ Parameter controlling the negative region
- $e \rightarrow$ Base of the natural logarithm

Working of ELU:

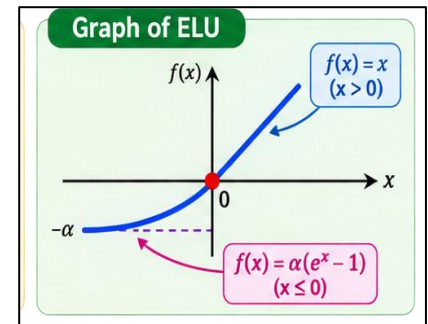
- For positive inputs, ELU behaves like ReLU.
- For negative inputs, ELU produces a negative value instead of zero.
- As x becomes more negative, the output approaches $-\alpha$.
- Therefore, ELU provides a smoother negative region than ReLU.

Advantages of ELU:

- Produces negative outputs for negative inputs.
- Maintains a non-zero gradient in the negative region.
- Helps reduce the Dying ReLU Problem.
- Provides a smoother transition around zero.
- Retains ReLU-like behavior for positive inputs.

Limitation of ELU:

- The exponential calculation for negative inputs makes ELU more computationally expensive.



4.2 SELU – Scaled Exponential Linear Unit:

SELU (Scaled Exponential Linear Unit) is a scaled version of the ELU activation function. It is designed to maintain the mean and variance of activations within a stable range during training.

This property is known as self-normalization.

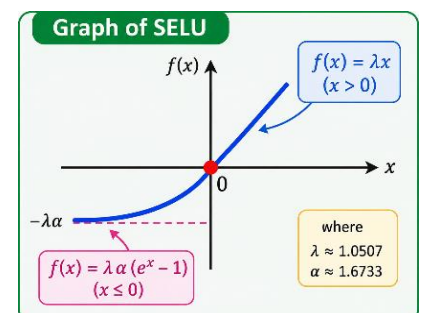
Formula
$$f(x) = \lambda \begin{cases} x, & x > 0 \\ \alpha(e^x - 1), & x \leq 0 \end{cases}$$

where:

- $x \rightarrow$ Input value
- $\alpha \rightarrow$ Controls the negative region
- $\lambda \rightarrow$ Scaling factor

Working of SELU:

- For positive inputs, SELU produces a scaled positive value.
- For negative inputs, it produces a scaled negative value.
- The scaling factor helps maintain stable activation statistics.



- SELU is designed to support self-normalizing neural networks.

Scaling Factor:

The scaling factor λ scales the output of the ELU function.

The standard SELU parameters are approximately:

$$\alpha \approx 1.6733 \quad \lambda \approx 1.0507$$

The scaling factor helps the activations maintain a stable mean and variance across layers.

Self-Normalizing Property:

In deep networks, the distribution of activations can change as information passes through layers. SELU is designed to automatically move the mean and variance of activations toward stable values under appropriate conditions.

Characteristics of SELU:

- It is based on the ELU function.
- Uses a scaling factor λ .
- Produces negative outputs for negative inputs.
- Supports self-normalizing behavior.
- Helps maintain stable activation statistics.

4.3 ReLU vs ELU vs SELU

Feature	ReLU	ELU	SELU
Positive input	x	x	λx
Negative input	0	Negative value	Negative value
Negative-region gradient	0	Non-zero	Non-zero
Main advantage	Simple & efficient	Reduces Dying ReLU problem	Self-normalizing behavior
Scaling factor	—	—	λ
Computational cost	Low	Higher than ReLU	Higher than ReLU
Key property	Sparse activation	Smooth negative region	Stable activation statistics

5. Unique Properties of CNN

CNNs have certain properties that make them particularly suitable for processing image data. These properties allow CNNs to extract useful features while reducing the number of parameters and computational requirements compared with fully connected networks.

The major unique properties of CNNs are:

5.1 Local Connectivity:

Local Connectivity means that a neuron in a convolution layer is connected only to a small local region of the input rather than to every pixel in the entire image.

- An image contains a large number of pixels.
- A CNN uses a small filter to examine only a limited region at a time.
- The filter focuses on local patterns such as: Edges , Lines, Corners, Textures
- The filter then moves to another region and performs the same operation.
- In this way, the network learns local features efficiently.

Example:

For a 3×3 filter, the convolution operation examines 3×3 pixels at a time rather than connecting a neuron to the entire image.

Importance of Local Connectivity:

- Reduces the number of connections.
- Reduces the number of parameters.
- Makes feature extraction computationally efficient.
- Helps CNNs identify local visual patterns.

5.2 Parameter Sharing:

Parameter Sharing is a property of CNNs in which the same filter (kernel) and its weights are used at different positions of the input image. Instead of learning a separate set of weights for every location, the CNN reuses the same filter across the image.

How Parameter Sharing Works:

- A convolution filter contains a fixed set of learnable weights.
- The filter is applied to one region of the input image.
- The same filter is then moved to other regions.
- The same weights are reused at every position.
- The filter therefore learns to detect the same type of feature wherever it appears in the image.

Example:

Suppose a filter learns to detect a vertical edge.

- The filter can detect a vertical edge on the left side of an image.
- The same filter can detect a vertical edge in the centre.
- It can also detect the same type of edge on the right side.

Thus, the CNN does not need a separate filter for each location.

Advantages of Parameter Sharing:

- **Reduces the number of parameters:** The same weights are reused across different locations.
- **Reduces memory requirements:** Fewer parameters need to be stored.

- **Improves computational efficiency:** The network does not need to learn separate weights for every image location.
- **Allows feature detection at different positions:** A learned feature can be detected wherever it occurs in the image.
- **Supports translation-related robustness:** The same feature can be recognized at different spatial locations.

5.3 Sparse Connectivity:

Sparse Connectivity is a property of CNNs in which each neuron is connected only to a small portion of the input, rather than being connected to every input value. This is achieved through the use of small convolution filters.

How Sparse Connectivity Works:

- A CNN uses a small filter to process a local region of the input.
- A neuron is connected only to the pixels within its receptive field.
- It does not need connections to all pixels in the image.
- The filter moves across the image and processes different local regions.
- This results in fewer connections compared with a fully connected network.

Example

Consider an image of size: 32×32

and a convolution filter of size: 3×3

A neuron associated with the filter processes only 9 input values at a particular location instead of all: $32 \times 32 = 1024$ pixels.

Thus, the neuron has a much smaller number of connections.

Advantages of Sparse Connectivity:

- Reduces the number of connections.
- Reduces the number of parameters.
- Reduces computational requirements.
- Makes feature extraction more efficient.
- Allows the network to focus on local patterns.

5.4 Hierarchical Feature Learning:

Hierarchical Feature Learning is the ability of a CNN to learn features at different levels of abstraction as the data passes through successive convolutional layers. Lower layers learn simple features, while deeper layers combine these features to learn more complex patterns.

How Hierarchical Feature Learning Works:

CNNs learn features progressively:

1. **Early Layers – Simple Features**
 - Learn basic visual patterns.
 - Examples: Edges, Lines, Corners
2. **Intermediate Layers – Combined Features**
 - Combine simple features learned by earlier layers.
 - Learn: Textures, Shapes, Simple object patterns
3. **Deeper Layers – Complex Features**

- Combine intermediate features.
- Learn: Object parts, Complex structures, High-level visual patterns

4. Final Layers – Object Representation

- Use the high-level features for tasks such as image classification or recognition.

Example:

For an image of a face, a CNN may learn:

Edges → Simple Shapes → Eyes/Nose/Mouth → Face Structure → Face Class

Thus, each deeper layer builds upon the representations learned by the preceding layers.

Importance of Hierarchical Feature Learning:

- Automatically learns features at different levels.
- Reduces the need for manual feature extraction.
- Allows simple features to be combined into complex representations.
- Helps CNNs recognize complex objects and patterns.
- Enables the network to learn meaningful representations directly from image data.

5.5 Preservation of Spatial Information:

Preservation of Spatial Information is an important property of CNNs that allows the network to retain information about the location and arrangement of features in an image while processing it.

In image data, the position of a feature is important. For example, in a face image, the relative positions of the eyes, nose, and mouth help the network understand the overall structure of the face.

How CNNs Preserve Spatial Information:

- Convolution operates on local regions of the input image.
- The filter moves systematically across the image.
- The resulting feature map maintains the spatial arrangement of detected features.
- Therefore, the network can identify where a particular feature occurs in the image.
- Pooling may reduce spatial dimensions, but it retains important feature responses.

Example:

Consider an image containing a vertical edge.

- If the edge appears on the left side, the corresponding feature map shows a strong response in that region.
- If the same edge appears on the right side, the feature map shows the response at the corresponding location.

Thus, CNNs retain useful information about both the feature and its location.

Importance:

- Helps CNNs understand the spatial arrangement of image features.
- Helps recognize objects based on the relationship between their parts.
- Supports effective processing of visual data.
- Enables the network to build meaningful higher-level representations from local

6. CNN Architectures

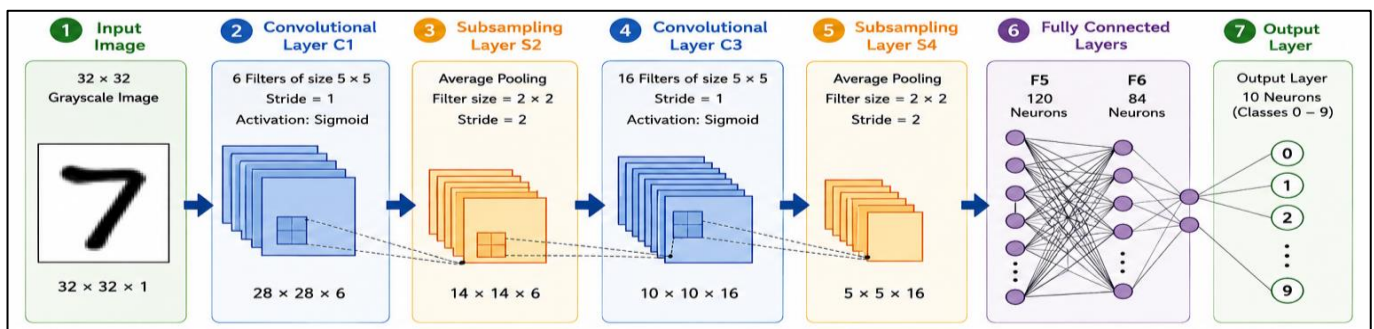
CNN architectures have evolved over time to improve feature extraction, classification accuracy, computational efficiency, and the ability to train deeper networks. Different architectures introduced different design ideas to overcome the limitations of earlier CNN models.

The important CNN architectures covered in this unit are: LeNet-5, AlexNet, VGG, ResNet

6.1 LeNet-5:

LeNet-5 is one of the early and influential Convolutional Neural Network architectures. It was designed primarily for handwritten digit recognition and demonstrated how convolution and pooling layers could be used effectively for image classification.

Architecture of LeNet-5:



1. Input Layer

- Receives the input image.
- The original LeNet-5 architecture uses a 32×32 grayscale image.

2. Convolution Layer (C1)

- Applies convolution filters to the input image.
- Extracts basic features such as edges and simple patterns.

3. Pooling/Subsampling Layer (S2)

- Reduces the spatial dimensions of the feature maps.
- Retains important feature information while reducing computational requirements.

4. Convolution Layer (C3)

- Performs another convolution operation.
- Learns more complex features from the representations produced by earlier layers.

5. Pooling/Subsampling Layer (S4)

- Further reduces the spatial dimensions.
- Provides more compact feature representations.

6. Fully Connected Layer (C5)

- Combines the learned features from the previous layers.
- Produces a high-level representation for classification.

7. Fully Connected Layer (F6)

- Further processes the learned representation.
- Provides information for the final classification.

8. Output Layer

- Produces the final classification result.
- For handwritten digit recognition, the output corresponds to the possible digit classes.

Important Features of LeNet-5:

- Designed for handwritten digit recognition.
- Uses convolution layers for feature extraction.
- Uses subsampling/pooling layers to reduce spatial dimensions.
- Uses fully connected layers for classification.
- Demonstrated the effectiveness of CNNs for image recognition.
- Established a basic architecture pattern that influenced later CNN architectures.

Significance of LeNet-5:

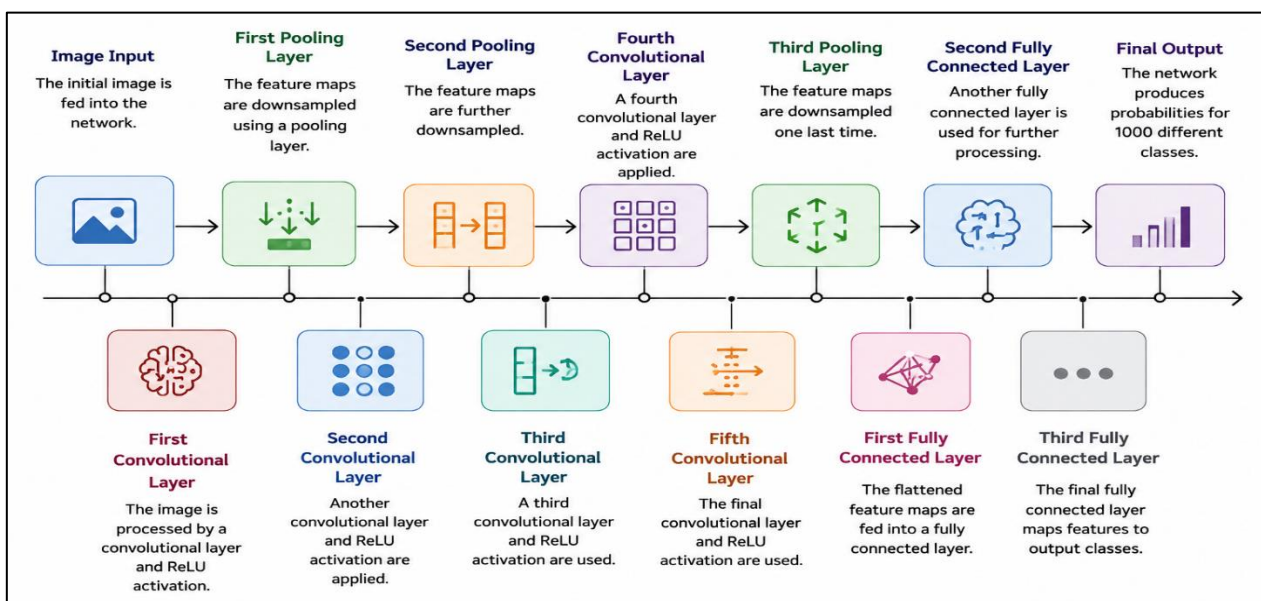
LeNet-5 is historically important because it demonstrated that a neural network could automatically learn useful visual features through convolution and use those features for classification. It became an important foundation for the development of more advanced CNN architectures.

6.2 AlexNet:

AlexNet is a deep Convolutional Neural Network architecture that achieved a major breakthrough in large-scale image classification. It demonstrated the effectiveness of deep CNNs, ReLU activation, GPU-based training, and data augmentation for complex image-recognition tasks.

Architecture of AlexNet:

AlexNet consists of a sequence of convolutional layers, ReLU activations, pooling layers, and fully connected layers, followed by the output layer.



1. Input Layer

- Receives the input image.
- AlexNet was designed for large-scale image classification.

2. Convolutional Layers

- Extract visual features from the input image.
- Earlier layers learn relatively simple features.
- Deeper convolutional layers learn more complex patterns.

3. ReLU Activation

- ReLU is applied after convolution.
- It introduces non-linearity and supports efficient training of the deep network.

4. Pooling Layers

- Reduce the spatial dimensions of feature maps.
- Help reduce computational requirements.

5. Fully Connected Layers

- Combine the features extracted by the convolutional layers.
- Produce high-level representations for classification.

6. Output Layer

- Produces the final classification result for the input image.

Important Features of AlexNet:

1. Use of ReLU Activation: AlexNet extensively used ReLU instead of traditional activation functions such as sigmoid and tanh.

- ReLU is computationally simple.
- It helps introduce non-linearity.
- It supports faster training of deep networks.

2. GPU-Based Training: AlexNet used Graphics Processing Units (GPUs) for training.

- GPUs provide high parallel computational capability.
- This made training large CNN models more practical.
- GPU-based deep learning became an important approach for training large neural networks.

3. Multiple Convolutional Layers: AlexNet used several convolutional layers to learn hierarchical visual features.

- Early layers → simple visual features
- Intermediate layers → textures and shapes
- Deeper layers → complex visual patterns

4. Pooling: Pooling layers were used to reduce the spatial dimensions of feature maps and reduce computational requirements.

5. Dropout: AlexNet used dropout in its fully connected layers to reduce overfitting.

Dropout temporarily disables some neurons during training, helping the network avoid depending too heavily on particular neurons.

6. Data Augmentation: AlexNet used data augmentation to increase the effective variety of training data.

Common transformations include:

- Image cropping
- Image flipping
- Other variations of training images

7. Large-Scale Image Classification: AlexNet was designed for large-scale image classification and demonstrated that deep CNNs could achieve strong performance on complex visual recognition tasks.

6.3 VGG (Visual Geometry Group) Network:

VGG (Visual Geometry Group) is a CNN architecture developed to investigate the effect of increasing the depth of a convolutional neural network. VGG is known for its simple and uniform architecture, mainly using small 3×3 convolution filters.

Basic Idea of VGG:

The main idea behind VGG is to build a deeper network by using a sequence of small convolution filters instead of using larger filters.

- Uses mainly 3×3 convolution filters.
- Uses multiple convolutional layers arranged in blocks.
- Uses ReLU activation after convolution.
- Uses max-pooling to reduce spatial dimensions.
- Uses fully connected layers toward the end for classification.
- Deeper VGG models can learn increasingly complex visual features.

Basic Architecture of VGG:

The convolution and pooling layers are arranged into blocks.

VGG Blocks:

- Block 1: Convolution layers → Max Pooling
- Block 2: Convolution layers → Max Pooling
- Block 3: Convolution layers → Max Pooling
- Block 4: Convolution layers → Max Pooling
- Block 5: Convolution layers → Max Pooling
- Fully Connected Layers: Classification
- Output Layer: Final prediction

As the network goes deeper:

- Spatial dimensions are progressively reduced through pooling.
- The number of feature channels generally increases.
- The network learns increasingly complex features.

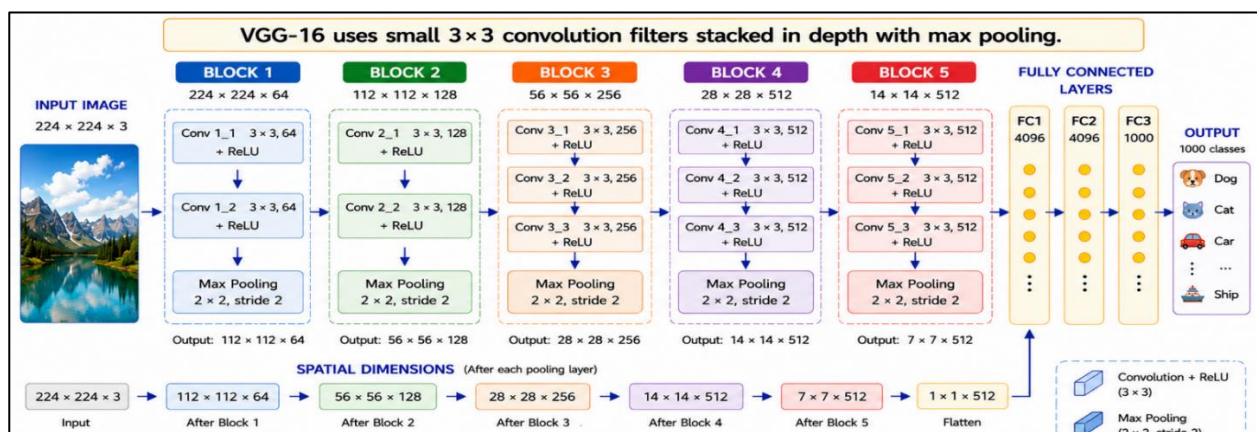
VGG-16:

VGG-16 is a VGG architecture containing 16 learnable layers.

Main Characteristics:

- Uses 3×3 convolution filters.
- Uses ReLU activation.
- Uses max-pooling layers.
- Contains multiple convolutional blocks.
- Uses fully connected layers toward the end.
- Provides a relatively simple and uniform deep CNN architecture.

Block	Conv. Layers	Filters	Output after Pooling
Block 1	2	64	$112 \times 112 \times 64$
Block 2	2	128	$56 \times 56 \times 128$
Block 3	3	256	$28 \times 28 \times 256$
Block 4	3	512	$14 \times 14 \times 512$
Block 5	3	512	$7 \times 7 \times 512$



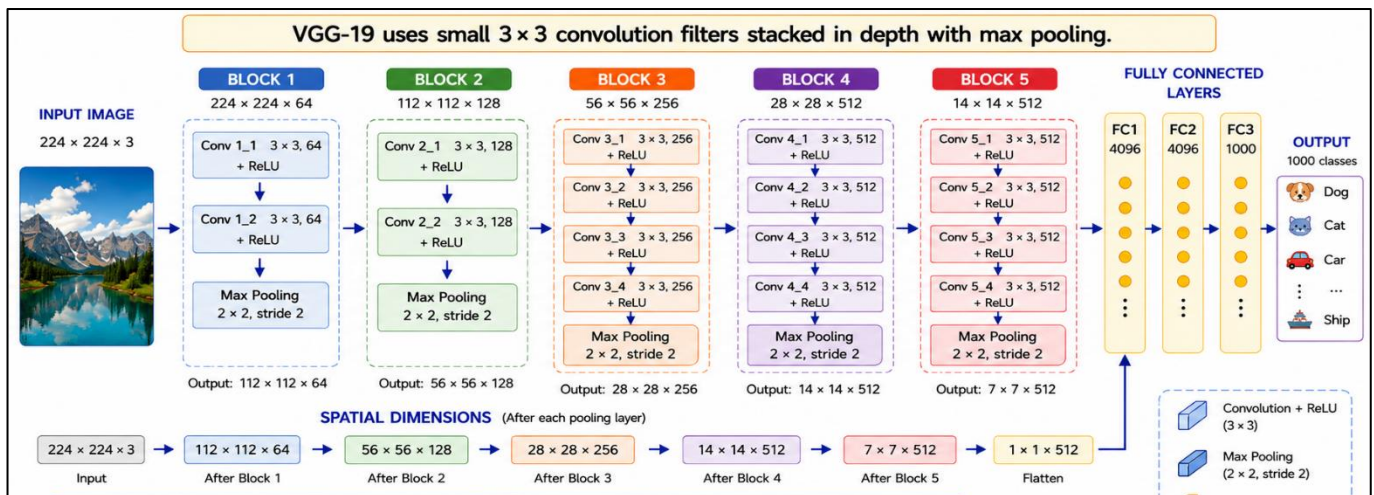
VGG-19:

VGG-19 is a deeper VGG architecture containing 19 learnable layers.

Main Characteristics

- Uses 3×3 convolution filters.
- Uses ReLU activation.
- Uses max-pooling.
- Contains more convolutional layers than VGG-16.
- Provides greater network depth for learning visual features.

Block	Conv. Layers	Filters	Output after Pooling
Block 1	2	64	$112 \times 112 \times 64$
Block 2	2	128	$56 \times 56 \times 128$
Block 3	4	256	$28 \times 28 \times 256$
Block 4	4	512	$14 \times 14 \times 512$
Block 5	4	512	$7 \times 7 \times 512$

**VGG-16 vs VGG-19:**

Feature	VGG-16	VGG-19
Learnable layers	16	19
Convolutional depth	Lower	Higher
Basic filter	3×3	3×3
Basic design	Convolution + ReLU + Pooling	Convolution + ReLU + Pooling
Network depth	Deep	Deeper
Complexity	Relatively lower	Relatively higher

6.4 ResNet (Residual Network):

ResNet (Residual Network) is a deep Convolutional Neural Network architecture designed to make the training of very deep networks easier and more effective. It introduced residual learning and skip connections, which allow information and gradients to pass through the network more effectively.

Need for ResNet:

Simply increasing the depth of a CNN does not always improve its performance. Very deep networks can face difficulties such as:

- **Vanishing gradients:** Gradients may become extremely small during backpropagation.
- **Degradation problem:** Adding more layers may cause training accuracy to stop improving or even decrease.
- **Difficulty in optimization:** Very deep networks can become difficult to train effectively.
- **Information flow problems:** Information may not pass efficiently through many successive layers.

ResNet addresses these difficulties by introducing shortcut/skip connections.

Residual Learning:

In a traditional network, a stack of layers attempts to directly learn the desired mapping: $H(x)$

ResNet instead learns the residual mapping:

$$H(x) = F(x) + x$$

where:

- $x \rightarrow$ Input to the residual block
- $H(x) \rightarrow$ Desired underlying mapping
- $F(x) \rightarrow$ Residual mapping learned by the stacked layers

The final output is:

$$y = F(x) + x$$

Main Idea:

The network does not need to learn the complete transformation $H(x)$. Instead, the convolutional layers learn the residual $F(x)$, while the original input x is directly added to it.

$$\text{Output} = \text{Residual Mapping} + \text{Input}$$

This residual-learning approach makes the optimization of deep networks easier.

Skip Connection:

A Skip Connection, also called a Shortcut Connection, provides a direct path through which the input can bypass one or more layers and reach a later layer.

Working:

1. The input x enters the residual block.
2. The input is passed through a sequence of convolutional layers to learn $F(x)$.
3. At the same time, the original input travels through the shortcut connection.
4. The residual output $F(x)$ and the shortcut output x are added.
5. The combined result is passed to the next stage.

Mathematically:

$$y = F(x) + x$$

Importance of Skip Connection:

- Provides a direct path for information flow.
- Helps gradients propagate through deep networks.
- Reduces the difficulty of training very deep networks.
- Helps address the degradation problem.
- Supports effective residual learning.

Residual Block:

A Residual Block is the basic building block of ResNet. It consists of convolutional layers and a shortcut connection.

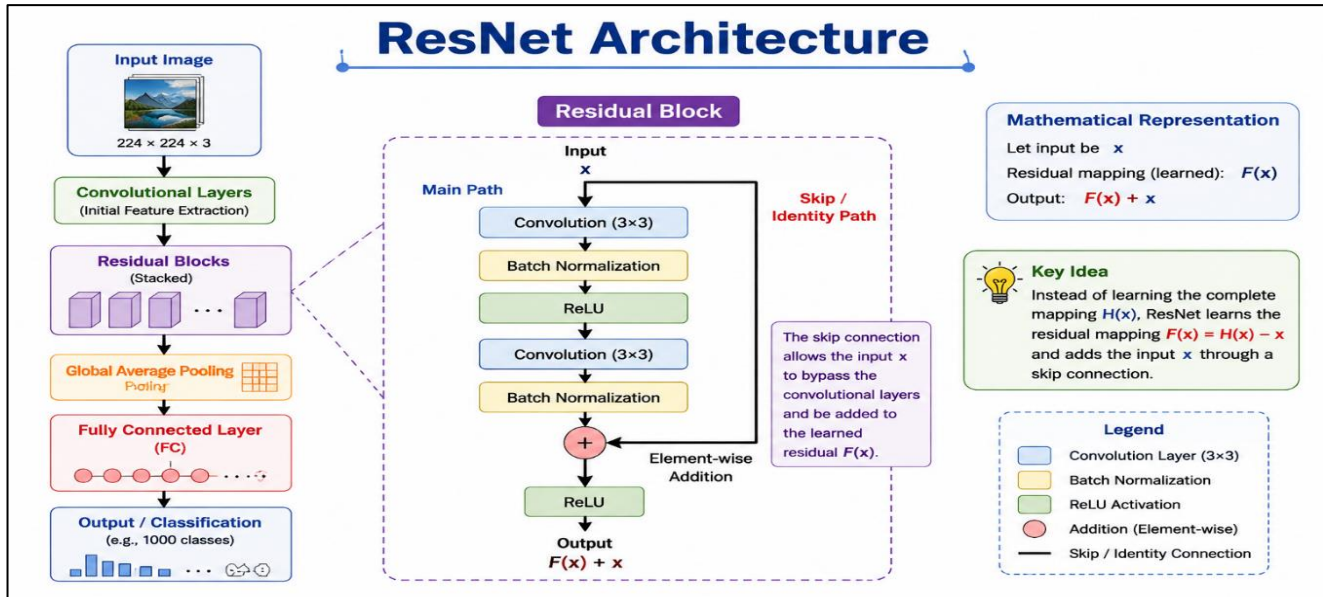
The main path learns: $F(x)$

The shortcut path carries: x

The two paths are added: $y = F(x) + x$

Working of a Residual Block:

1. Input x is given to the residual block.
2. The input passes through convolutional layers.
3. The layers learn the residual mapping $F(x)$.
4. The original input x passes through the shortcut connection.
5. $F(x)$ and x are added.
6. The resulting output is passed to the next layer.



ResNet Versions:

Different versions of ResNet were developed by increasing the depth of the network. The commonly used versions include: ResNet-18, ResNet-34, ResNet-50, ResNet-101, ResNet-152

ResNet Version	Approximate Depth
ResNet-18	18 layers
ResNet-34	34 layers
ResNet-50	50 layers
ResNet-101	101 layers
ResNet-152	152 layers

Basic Residual Blocks and Bottleneck Blocks:

- ResNet-18 and ResNet-34 mainly use the basic residual block.
- ResNet-50, ResNet-101, and ResNet-152 use the bottleneck block.

As the depth increases, the network can learn more complex representations. However, simply adding more layers also increases the computational cost and number of parameters.

Need for Bottleneck Block:

- As the depth of ResNet increases, the number of convolutional operations and computational requirements also increase.
- Using the basic residual block repeatedly in very deep networks can become computationally expensive, especially when the number of feature channels is large.
- Therefore, deeper ResNet versions use a Bottleneck Block to reduce computation while maintaining the residual-learning approach.

Why is Bottleneck Needed?

- Deeper networks require more computations.
- A 3×3 convolution becomes expensive when the number of channels is large.
- The number of channels can first be reduced before applying the 3×3 convolution.
- After feature extraction, the number of channels can be expanded again.
- This makes very deep ResNet architectures more computationally efficient.

Basic Idea

Reduce Channels $\rightarrow$ Feature Extraction $\rightarrow$ Restore Channels

This leads to the Bottleneck Block, which uses:

$1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1$

The detailed structure and working of the Bottleneck Block are discussed next.

Bottleneck Block:

A Bottleneck Block is a special type of residual block used in deeper ResNet architectures such as ResNet-50, ResNet-101, and ResNet-152.

It generally contains three convolutional layers:

1×1 Convolution

- Reduces the number of feature channels.
- Decreases computational cost.

3×3 Convolution

- Performs the main spatial feature extraction.
- Learns features from the reduced representation.

1×1 Convolution

- Expands the number of feature channels.
- Restores the required channel dimension.

Basic Structure:

$x \rightarrow 1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1 \rightarrow F(x)$

The shortcut carries the input x , and the two paths are combined:

$y = F(x) + x$

Example:

Suppose the input contains 256 feature channels.

The bottleneck may transform the channels as:

$256 \rightarrow 64 \rightarrow 64 \rightarrow 256$

- $256 \rightarrow 64$: Channel reduction
- $64 \rightarrow 64$: Feature extraction using 3×3 convolution
- $64 \rightarrow 256$: Channel expansion

Why is it Called a Bottleneck?

- The first 1×1 convolution creates a narrower representation before the 3×3 convolution.
- Therefore, the computationally expensive 3×3 convolution operates on fewer channels.

Advantages of ResNet:

- Enables training of very deep CNNs.
- Uses skip connections to improve information flow.
- Helps reduce the effect of vanishing gradients.
- Helps address the degradation problem.
- Makes optimization of deep networks easier.
- Provides strong performance in image-recognition tasks.

Comparison of CNN Architectures:

Feature	LeNet-5	AlexNet	VGG	ResNet
Main purpose	Handwritten digit recognition	Large-scale image classification	Deep visual feature learning	Training very deep networks
Network depth	Relatively shallow	Deeper than LeNet-5	Deeper	Very deep
Important idea	Early practical CNN architecture	ReLU, GPU training, dropout, data augmentation	Small 3×3 filters and increased depth	Residual learning and skip connections
Activation	Traditional activation functions	ReLU	ReLU	ReLU
Pooling	Subsampling/pooling	Pooling	Max pooling	Pooling and residual blocks
Main architectural feature	Convolution + subsampling	Deep CNN with GPU-based training	Uniform stacked convolution blocks	Residual blocks
Skip connections	No	No	No	Yes
Bottleneck block	No	No	No	Used in deeper versions
Representative versions	LeNet-5	AlexNet	VGG-16, VGG-19	ResNet-18, 34, 50, 101, 152

7. Applications of CNN

Convolutional Neural Networks are widely used in applications where the input contains visual patterns and spatial information. Their ability to automatically learn features from images makes them suitable for many real-world applications.

1. Image Classification

- Classifies an image into predefined categories.
- Example: identifying an image as a cat, dog, car, or other object.

2. Object Detection

- Identifies objects present in an image and determines their locations.
- Used in surveillance, autonomous vehicles, and traffic monitoring.

3. Face Recognition

- Extracts facial features and identifies or verifies a person.
- Used in security and authentication systems.

4. Medical Image Analysis

- Helps analyze medical images such as X-rays, CT scans, and MRI images.
- Can assist in detecting abnormalities and diseases.

5. Image Segmentation

- Divides an image into meaningful regions or objects.
- Used in medical imaging, autonomous driving, and image processing.

6. Handwritten Character Recognition

- Recognizes handwritten digits and characters.
- **LeNet-5** is a well-known early CNN architecture developed for handwritten digit recognition.

7. Autonomous Vehicles

- Processes camera images to identify:
 - Vehicles
 - Pedestrians
 - Road signs
 - Lane markings
- Supports environment perception for autonomous driving systems.

8. Object and Scene Recognition

- Identifies objects and understands different visual scenes.
- Used in image search and intelligent vision systems.

Importance of CNNs in These Applications

CNNs are particularly useful because they can:

- Automatically learn relevant features.
- Preserve spatial relationships between image features.
- Detect patterns at different levels of complexity.
- Process large and complex image datasets.
- Reduce the need for manual feature extraction.